AI Candidate-Ranking System

Intelligent Candidate Discovery & Ranking Challenge

Team kafka_consumer

github.com/ojas4414/IndiaRun | Sandbox: huggingface.co/spaces/ojaasgandu/India_Run

The LLM should understand. It should never decide.

A deterministic retrieve-rerank pipeline with contrastive attention,
behavioral-twin disambiguation, and a measured, ablation-backed evaluation.

Agenda

What this deck covers

01 The problem & why it's hard

02 Thesis & system architecture

03 How every trap in the brief is defended against

04 Two novel components: contrastive attention, twin disambiguation

05 Evaluation methodology & measured results (baselines, ablation, adversarial)

06 Sample output, guarantees, honest limitations, and tech stack

The Problem

What we were asked to solve

- Rank the top 100 of 100,000 candidates for one Senior AI Engineer JD (Redrob AI, Series A).
- "The right answer is not: find candidates whose skills section contains the most AI keywords." — that trap is explicitly built into the dataset.
- Traps: keyword stuffers, plain-language strong fits ("Tier-5s"), behavioral twins, ~80 honeypots (impossible profiles). Honeypot rate >10% in the top-100 is an automatic disqualification.
- Constraints: <=5 min, 16 GB RAM, CPU-only, no network during ranking; output must be reproducible.

Why this is hard

Precision, adversarial data, and a hard runtime budget, all at once

- Precision vs. constraints: the highest-precision approach (an LLM judging every profile) is exactly what the time/CPU/offline budget rules out.
- The dataset is adversarial by design: strong keyword overlap correlates with rank in a naive system, but is explicitly NOT what the JD wants.
- The JD itself asks for hiring judgement most systems never encode: availability, tenure patterns, verified vs. self-reported skill, and production vs. tutorial-level experience.

Thesis

The LLM should understand, never decide

- Letting an LLM rank is non-deterministic, slow, and can't run offline in budget.
- So we split UNDERSTANDING (offline embeddings) from RANKING (deterministic scoring).
- rank.py makes zero LLM/network calls and returns byte-identical output on identical input.
- Everything that needs judgement is pushed into precomputed features and explicit scoring rules -- never into a runtime model call.

Pipeline: retrieve -> rerank -> disambiguate

Architecture

- Precompute (no time limit): honeypot filter -> sentence-transformer embeddings (all-MiniLM-L6-v2) -> FAISS index (~47 min for 100k candidates, one-time cost).
- Stage 1 Retrieval: FAISS ANN -> top-2000 semantic funnel (recall, not final ranking).
- Stage 2 Feature rerank: trajectory-DP 35% | skill-graph 22% | behavioral 18% | attention 15% | semantic 10% - disqualifier penalties.
- Stage 3 Attention rerank over the top-600 survivors (precision + evidence extraction).
- Stage 4 Behavioral-twin disambiguation -> top-100 + evidence-grounded reasoning (~40s CPU total).

Design decisions: why these weights

Every weight in scoring/hybrid_scorer.py traces back to a measurement

- Trajectory carries the most weight (35%) because the ablation shows it is the single strongest JD-fit signal -- removing it drops NDCG@100 from 0.876 to 0.753.
- Semantic similarity is weighted lowest (10%): it is mainly a RECALL signal for the funnel, and the ablation shows it is nearly inert in final ordering once the funnel already exists.
- Behavioral (18%) and attention (15%) are kept as deliberate minority signals: they optimize availability and explainability, which the JD explicitly requires but a pure fit-rubric can't see.
- Weights were tuned FROM the evaluation harness, not chosen first and rationalized after.

How each trap is handled

Trap defenses map directly to the brief

- Keyword stuffing -> skill-graph + trajectory carry 57% of weight; off-domain titles hard-disqualified before they ever reach the scorer.
- Plain-language Tier-5s -> semantic retrieval + sentence attention surface hidden production evidence even when a profile never uses the JD's exact vocabulary.
- Behavioral twins -> cluster near-duplicates by role/experience/skill-Jaccard; the most-available twin leads, the rest are demoted so duplicates don't stack the top-100.
- ~80 honeypots -> date/tenure impossibility checks at precompute (52 caught in this run; 0 in top-100, vs. the 10% disqualification threshold).
- Consulting-only / CV-speech-only / title-chasers / no-recent-code -> explicit disqualifier rules lifted directly from the JD's "things we explicitly do NOT want" section.

Novel #1: contrastive-facet sentence attention

Precision + explainability

- Cross-attention pooling: JD facets = queries, candidate sentences = keys/values (ColBERT-style MaxSim / late interaction).
- Recovers a single strong sentence that document-level embedding would average away -- e.g. one line about shipping a production retrieval system, buried in an otherwise generic profile.
- Contrastive anti-fit facets cancel sentence-level keyword stuffing (e.g. "SEO articles that ranked in search" superficially resembles retrieval language but nets to zero against the anti-fit facet).
- The highest-attention sentence is surfaced as a literal quoted evidence line in the candidate's generated reasoning -- not just a score, a citation.

Novel #2: twins + adversarial robustness

Things a cosine-similarity submission won't have

- Behavioral-twin disambiguation: paper-identical profiles (same role, experience, skills) are separated by availability signals instead of being ranked as duplicates.
- Adversarial test: inject every JD skill + fake 95/100 assessment scores + a buzzword summary into profiles that should NOT rank, and measure promotion into the top-100.
- Off-domain promotion under attack: FULL system 0% vs naive keyword ranker 100%.
- We also report an honestly-found residual weakness (generic engineers under extreme stuffing) plus its mitigation (skill grounding) -- rigor over spin.

Evaluation methodology

Evaluation the JD explicitly asks for

- The JD explicitly lists NDCG / MRR / MAP / offline-to-online correlation as required skills for this role -- so the submission itself needed to demonstrate that rigor, not just claim it.
- Two kinds of evidence: (1) NDCG@10/@100, MRR, MAP@100 against a transparent recruiter rubric (weak supervision, documented circularity caveat); (2) label-free trap metrics -- honeypot % and off-domain-title % in the top-100 -- which need no labels at all.
- Compared against two baselines (naive keyword count, pure semantic similarity) and a full per-component ablation (drop skill-graph / trajectory / behavioral / attention / semantic one at a time).
- Plus the adversarial keyword-stuffing test above, targeting the exact trap the JD calls out.

Results: baselines vs. full system

Full system vs. naive-keyword and pure-semantic baselines

config	NDCG@10	NDCG@100	MRR	MAP@100	HP%	OffDom%
naive_keyword	0.653	0.615	0.500	0.790	0.0	6.0
semantic_only	0.656	0.508	1.000	0.789	0.0	27.0
full system	**1.000**	**0.876**	**1.000**	**0.985**	**0.0**	**4.0**

Off-domain titles in the top-100 drop from 27% (semantic-only) to 4% (full system). Honeypots stay at 0% for every gated config -- well under the 10% disqualification bar.

Results: per-component ablation

Each row removes one signal from the full system

config	NDCG@10	NDCG@100	MRR	MAP@100
full (all signals)	1.000	0.876	1.000	0.985
- attention	1.000	0.938	1.000	0.997
- skill_graph	1.000	0.898	1.000	0.992
- trajectory	**0.879**	**0.753**	**1.000**	**0.916**
- behavioral	1.000	0.904	1.000	0.989
- semantic	1.000	0.884	1.000	0.984

Removing trajectory causes by far the largest drop (0.876 -> 0.753) -- confirming it as the dominant JD-fit signal. Attention/behavioral/semantic optimize objectives (explainability, availability, recall) the rubric doesn't score, which is why removing them can nudge rubric-NDCG up.

Results: adversarial keyword-stuffing attack

Every JD skill + fake 95/100 assessments injected into profiles that shouldn't rank

cohort	n	full system	naive keyword
off-domain roles	**40**	**0%**	**100%**
generic engineers	4	100%	100%

Off-domain roles are fully robust: the disqualifier gate keys on the (unchanged) title, and trajectory keys on the (unchanged) career history, so stuffing changes nothing. Residual weakness (generic engineers, n=4): claimed-but-uncorroborated skills can still promote a borderline profile -- honestly reported, with skill-grounding logged as the fix.

Top-100 composition (current run)

Every one of the top 100 holds an on-domain AI/ML/Search title

count	current title
17	Search Engineer
12	Applied ML Engineer
12	AI Engineer
11	Machine Learning Engineer
8	AI Research Engineer
6	Senior NLP Engineer
6	Senior Data Scientist
5	Senior Machine Learning Engineer
4	Recommendation Systems Engineer
4	Staff Machine Learning Engineer
3	Senior AI Engineer
3	NLP Engineer
3	ML Engineer
2	Senior Applied Scientist

Sample output (real, from this run's submission.csv)

Every reasoning string is unique and cites a real profile quote

#1 score 0.7879 — Senior Machine Learning Engineer

Standout candidate -- 7.2 years of work as Senior Machine Learning Engineer. Brings Weaviate, Pinecone, Information Retrieval, Milvus -- the core of the embeddings & retrieval stack this role centres on. Assessment-verified in Weaviate (72/100).

#2 score 0.7463 — Staff Machine Learning Engineer

Excellent match -- Staff Machine Learning Engineer, ~7 yrs experience. Brings QLoRA, Pinecone, BM25, Information Retrieval. Evidence: "Spent substantial time on the boring-but-critical parts: incremental index refresh, embedding drift..."

#3 score 0.7441 — Lead AI Engineer

Strong hire signal -- Lead AI Engineer with 6.7 yrs experience. Hands-on with Information Retrieval, Learning to Rank, Elasticsearch, Python. From their profile: "Designed the offline evaluation framework from scratch -- NDCG, MRR, recall@K calibrated against online A/B metrics."

Results & guarantees

Meets every hard constraint

- Ranks the full 100k in ~40s (limit: 5 min); 0 honeypots and ~100% on-domain titles in the top-100.
- Deterministic: two independent runs produce byte-identical output (MD5-verified).
- Offline: embedding model baked into the Docker image; ranking sets TRANSFORMERS_OFFLINE=1 and HF_HUB_OFFLINE=1, no network calls at rank time.
- Reproducible end-to-end: `docker compose up --build` runs precompute -> rank -> validate in one command.
- 24 automated tests covering the skill graph, trajectory DP, honeypot rules, attention math, twin clustering, and evaluation metrics.

Honest limitations

What we'd fix next, said plainly

- The recruiter-rubric NDCG (gold.py) is weak supervision that partially overlaps with the ranker's own features (title, career text) -- read it as "agreement with an explicit rubric," not an unbiased oracle. The label-free HP%/OffDom% metrics are the load-bearing evidence.
- Attention contributes only 15% of the score by design; its main value is the evidence quote in each reasoning, not raw NDCG -- a deliberate trade for explainability the rubric can't measure.
- The adversarial test found a residual gap: heavily skill-stuffed generic engineers can still be promoted. Logged as future work (skill grounding: discount claims never corroborated in career text).
- We chose to report this rather than hide it -- an adversarial test is only useful if you act on what it finds.

Tech stack

Deliberately dependency-light and fully inspectable

- Embeddings & retrieval: sentence-transformers (all-MiniLM-L6-v2), FAISS (IndexFlatIP / cosine).
- Scoring: pure Python -- skill-graph BFS, Needleman-Wunsch trajectory alignment, rule-based disqualifiers and behavioral scoring, cross-attention pooling with contrastive facets.
- Packaging: Docker + docker-compose (one-command reproduction), pytest (24 tests), Gradio sandbox on HuggingFace Spaces for live, small-sample demos.
- Evaluation: a custom harness (NDCG/MRR/MAP, ablation, adversarial attack) -- no external eval library needed, everything is inspectable in evaluation/.

Thank you

Team kafka_consumer

Code: github.com/ojas4414/IndiaRun

Sandbox: huggingface.co/spaces/ojaasgandu/India_Run

Happy to walk through any design decision in detail.